

Project Proposal: Autonomous Intersection Management Using Critical-Zone Time-Slot Reservation

Group A / Team A9

June 15, 2026

Purpose of this Document

This document proposes a shared system concept for our autonomous intersection project. The goal is to align the whole team before we update the requirements diagram, use-case diagram, activity diagram, sequence diagram, UPPAAL timed automata model, and later the FreeRTOS / HW-SW implementation. The main problem we are solving is not simply “which car goes first?” but rather how autonomous vehicles can safely and efficiently share an intersection that contains multiple conflict areas.

1 Core System Idea

We model the autonomous intersection as a real-time shared-resource system. The intersection is the shared resource, vehicles are concurrent processes, and the conflict areas inside the intersection are critical zones. A vehicle may cross the intersection only according to a reservation schedule assigned by the controller.

Main safety rule: No two vehicles may occupy or reserve the same critical zone during overlapping time intervals.

This means the system is not limited to allowing only one vehicle to cross. Multiple vehicles may cross simultaneously if their required critical-zone sequences do not conflict in time.

1.1 Operating-System Analogy

Operating system concept	Intersection concept
Process / task	Vehicle
Shared resource	Intersection
Critical section	Critical zone inside the intersection
Scheduler	Intersection controller
Ready queue	Lane queue / waiting vehicles
Resource allocation	Critical-zone time-slot reservation
Deadline / slack time	Latest smooth arrival time before stopping
Deadlock	Vehicles blocking each other inside the intersection

2 Proposed Intersection Structure

The model should distinguish between the approach area and the actual conflict area. Vehicles are detected before they reach the dangerous part of the intersection. In this approach zone, the controller can manage speed so that vehicles reach the first critical zone at the correct time.

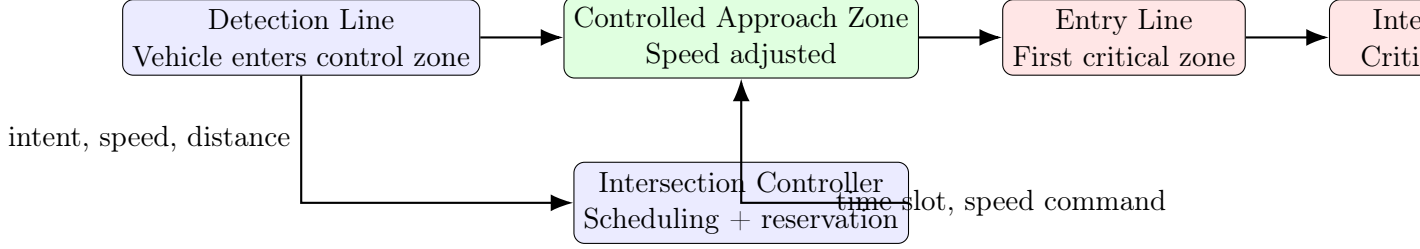
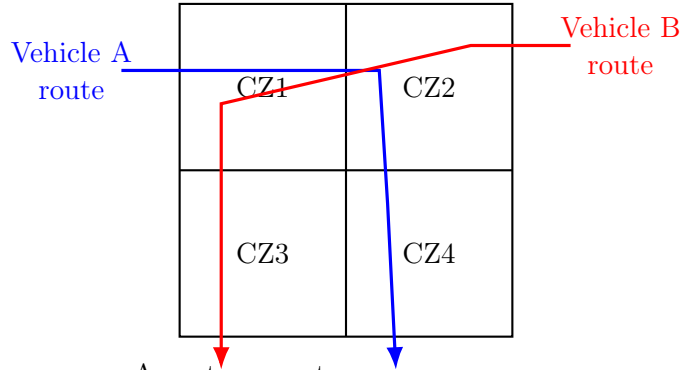


Figure 1: High-level concept: vehicles are controlled before they enter the critical zones.

3 Critical-Zone Model

The intersection should be divided into smaller critical zones. A route is converted into a sequence of zones that the vehicle must reserve. The exact number and shape of zones can be simplified for UPPAAL, but the principle should remain consistent.



A route maps to a zone sequence.

Reservations must not overlap in the same zone and time slot.

Figure 2: Example critical-zone abstraction. The final zone layout can be adjusted to match our intersection geometry.

4 Vehicle Arrival and Speed Control Assumption

To keep the model understandable, we assume each head-of-line vehicle enters the control zone at a fixed distance with a known starting speed. This makes arrival-time prediction simple enough for a timed automata model.

Let:

- D be the fixed distance from the detection line to the first critical-zone entry line,
- v_0 be the starting speed when detected,
- $v_{max} = 2v_0$ be the maximum allowed speed,
- v_{min} be the minimum smooth speed before the vehicle must stop.

The controller can compute the reachable arrival window:

$$T_{earliest} = \frac{D}{v_{max}}, \quad T_{normal} = \frac{D}{v_0}, \quad T_{latest} = \frac{D}{v_{min}}.$$

The controller searches for the earliest non-conflicting critical-zone schedule between $T_{earliest}$ and T_{latest} . If no schedule exists within this window, the vehicle should stop before entering the intersection.

5 Time-Slot Reservation Concept

Each vehicle receives a schedule for its required zone sequence. A vehicle does not need all zones at the same instant. It only needs each zone during the specific interval in which it passes through that zone.

Zone	t=10	t=11	t=12	t=13	t=14	t=15
CZ1	Car A	–	Car C	–	–	–
CZ2	–	Car B	–	Car C	–	–
CZ3	–	Car A	–	–	Car D	–
CZ4	Car B	–	–	Car D	–	–

Figure 3: Example reservation table. A zone may be used by only one vehicle in each time slot.

6 Scheduling and Priority Policy

Only the first vehicle in each incoming lane is active. Since there are four incoming lanes, the controller compares at most four active vehicles at a time. Vehicles behind the first vehicle in a lane cannot request access before the lane leader has moved.

The controller should use a simple and explainable priority policy:

1. Emergency vehicles have highest priority.
2. Normal vehicles are prioritized by smallest slack time.
3. If slack is similar, the vehicle with longer waiting time has priority.
4. If still equal, use FIFO order.
5. Non-conflicting vehicles may be scheduled together to improve throughput.

Slack time can be understood as:

$$slack = T_{latest} - T_{assigned_entry}.$$

A small slack means the vehicle is close to the point where it can no longer cross smoothly without stopping. However, priority never overrides safety. A vehicle may only receive a slot if its required critical zones are free during the assigned intervals.

7 Controller Algorithm

8 Communication Roles: V2I and V2V

V2I is the main scheduling communication path. Vehicles send requests to the controller, and the controller sends schedules back to the vehicles. V2V is not the main scheduler; it supports safety awareness.

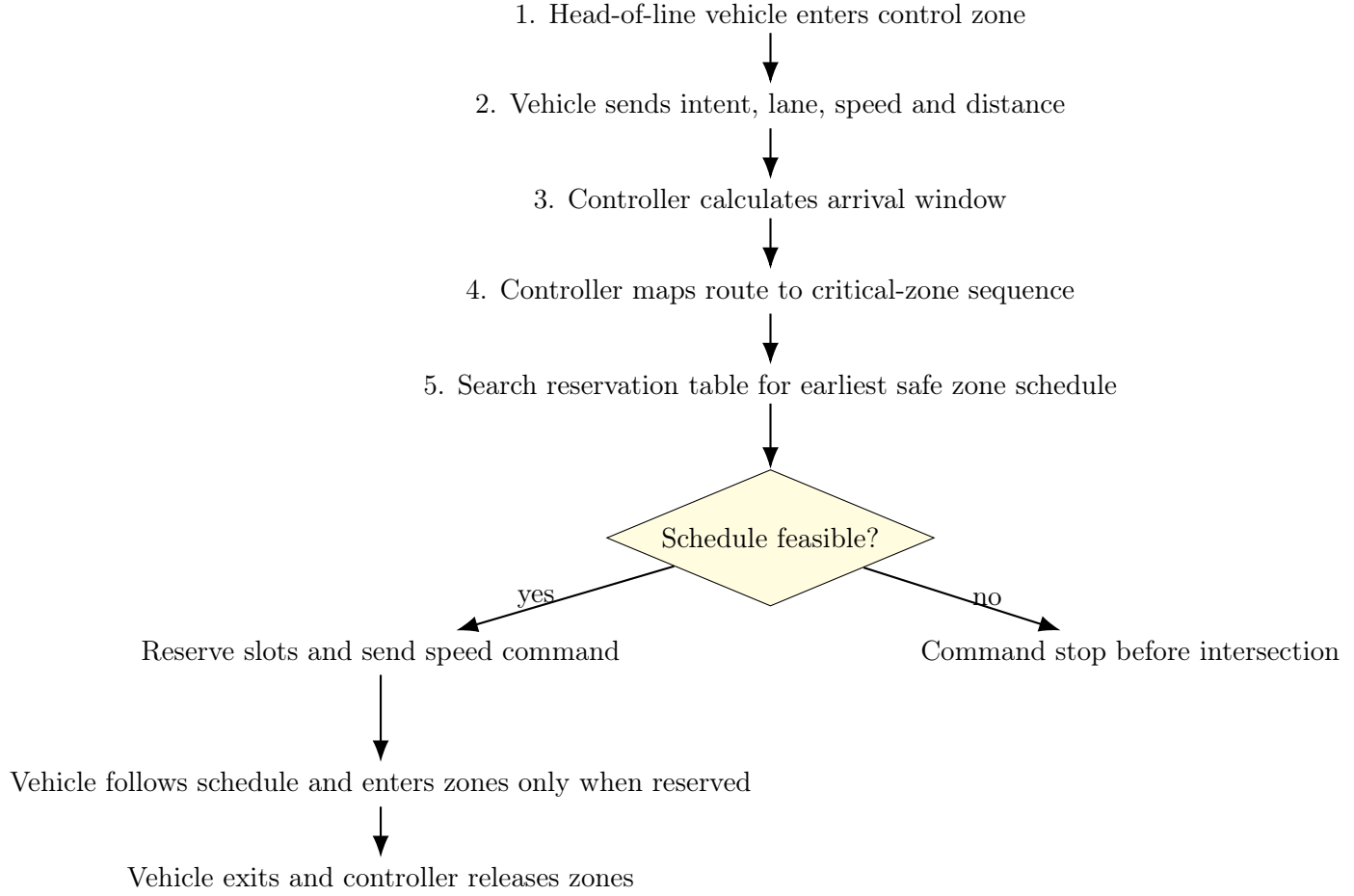


Figure 4: Proposed controller activity flow.

Communication	Role in our model
V2I	Vehicle sends route intent, speed, position and lane to the controller. Controller returns zone schedule and speed command.
V2V	Vehicle broadcasts intent, emergency state or warning information to nearby vehicles for awareness and fail-safe behavior.

9 Updated Requirement Set

ID	Requirement
REQ-1	The system shall detect vehicles before they enter the intersection.
REQ-2	Only the head-of-line vehicle in each incoming lane may request access to the intersection.
REQ-3	A vehicle shall declare its route intent before reaching the critical zones.
REQ-4	The controller shall map each vehicle route to a sequence of required critical zones.
REQ-5	The controller shall calculate a reachable arrival window using fixed distance, starting speed, maximum speed and minimum smooth speed.

REQ-6	The controller shall assign non-overlapping time slots for each required critical zone.
REQ-7	A vehicle may enter a critical zone only during its assigned reservation interval.
REQ-8	No two vehicles may reserve or occupy the same critical zone during overlapping time intervals.
REQ-9	Vehicles with non-conflicting critical-zone schedules may cross simultaneously.
REQ-10	The controller shall use speed commands to help vehicles reach their assigned entry time smoothly.
REQ-11	The controller shall prioritize vehicles using emergency status, slack time, waiting time and FIFO order.
REQ-12	If no feasible smooth crossing schedule exists, the vehicle shall stop before the first critical zone.
REQ-13	The UPPAAL model shall verify collision freedom, reservation validity, deadlock freedom and relevant timing constraints.

10 Suggested UML Updates

10.1 Use-Case Diagram

The updated use-case diagram should include the following use cases:

- Detect approaching vehicle
- Declare vehicle intent
- Calculate arrival window
- Map route to critical zones
- Reserve critical-zone time slots
- Send speed command
- Enter critical zone according to schedule
- Stop before intersection if no feasible slot exists
- Broadcast intent or emergency state through V2V

10.2 Activity Diagram

The activity diagram should follow the controller algorithm shown in Figure 4. The main decision point should be: *Is a non-conflicting and reachable schedule available?*

10.3 Sequence Diagram

The updated sequence diagram should include these lifelines:

- Vehicle
- Vehicle local controller
- Intersection controller
- Reservation table
- Critical zones
- Other vehicles for V2V awareness

Important messages:

- Vehicle → Controller: request(route, speed, distance, lane)
- Controller → Reservation table: checkAvailable(routeZones, arrivalWindow)

- Reservation table → Controller: earliest feasible schedule
- Controller → Vehicle: schedule(zoneTimes, speedCommand)
- Vehicle → Critical zone: enter only if reservation is valid
- Vehicle → Controller: zone exited / crossing complete

11 UPPAAL Modelling Plan

UPPAAL should model a network of timed automata. The first improved version should stay abstract and verifiable instead of modelling full vehicle physics.

11.1 Templates

- **Vehicle template:** Idle, Detected, Requesting, ControlledApproach, WaitingBeforeEntry, InCriticalZone, Exited.
- **Controller template:** ReceiveRequest, CalculateWindow, MapRoute, FindSlot, ReserveSlots, SendCommand, EmergencyMode.
- **Reservation table logic:** Abstract arrays storing zone ownership per time slot.
- **Optional emergency template:** Creates emergency request and override condition.

11.2 Discrete Speed Modes

Instead of continuous acceleration, use discrete speed modes:

Mode	Meaning
FAST	Reach first critical zone at earliest arrival time.
NORMAL	Reach first critical zone at normal arrival time.
SLOW	Reach first critical zone near latest smooth arrival time.
STOP	Stop before the first critical zone if no feasible slot exists.

11.3 Properties to Verify

- No two vehicles occupy the same critical zone at the same time.
- A vehicle enters a critical zone only if it has a valid reservation.
- The system is deadlock-free.
- Emergency priority does not create unsafe conflicts.
- A vehicle that cannot be scheduled smoothly stops before the intersection.

Example UPPAAL query ideas:

```
A[] not deadlock
A[] no_two_vehicles_same_zone
A[] vehicle_in_zone imply reservation_valid
A[] emergency_active imply no_conflicting_normal_vehicle_enters
```

12 Consistency Checklist

Before submitting updated work, check that all diagrams and text use the same concepts:

- Are we using “critical zone” consistently instead of vague “intersection area”?
- Does every route map to a sequence of critical zones?
- Does the controller assign time slots, not just permission?

- Does speed control happen before the first critical zone?
- Is stopping only allowed before the intersection, not inside a critical zone?
- Is V2I the main scheduling path and V2V the supporting safety path?
- Are priority rules safety-preserving and deadline-aware?
- Does the UPPAAL model verify the same requirements shown in the diagrams?

13 Related Work to Add to Zotero / BibTeX

The following papers are useful starting points for the report and final justification:

- Dresner and Stone introduced reservation-based autonomous intersection management using an intersection manager and vehicle requests.
- Miculescu and Karaman connect autonomous intersection coordination with polling and queueing ideas, including speed adjustment and safety guarantees.
- Hu, Smith and Goldstein discuss schedule-driven intersection control where approaching vehicle arrival times can be adjusted through velocity control.

References

- [1] K. Dresner and P. Stone, “Multiagent Traffic Management: A Reservation-Based Intersection Control Mechanism,” AAMAS, 2004.
- [2] K. Dresner and P. Stone, “A Multiagent Approach to Autonomous Intersection Management,” Journal of Artificial Intelligence Research, vol. 31, pp. 591–656, 2008.
- [3] D. Miculescu and S. Karaman, “Polling-systems-based Autonomous Vehicle Coordination in Traffic Intersections with No Traffic Signals,” arXiv:1607.07896, 2016.
- [4] H.-C. Hu, S. F. Smith and R. Goldstein, “Cooperative Schedule-Driven Intersection Control with Connected and Autonomous Vehicles,” IEEE/RSJ International Conference on Intelligent Robots and Systems, 2019.

Proposed Team Decision

The team should agree that our improved model is a critical-zone time-slot reservation system with speed-controlled arrival. This gives us a clear structure for requirements, UML diagrams, UPPAAL verification and implementation mapping. It also gives us a strong explanation of why multiple vehicles can cross at the same time while still guaranteeing safety.